



Exceptions

R. Ferrero

Politecnico di Torino

Dipartimento di Automatica e Informatica (DAUIN)

Torino - Italy

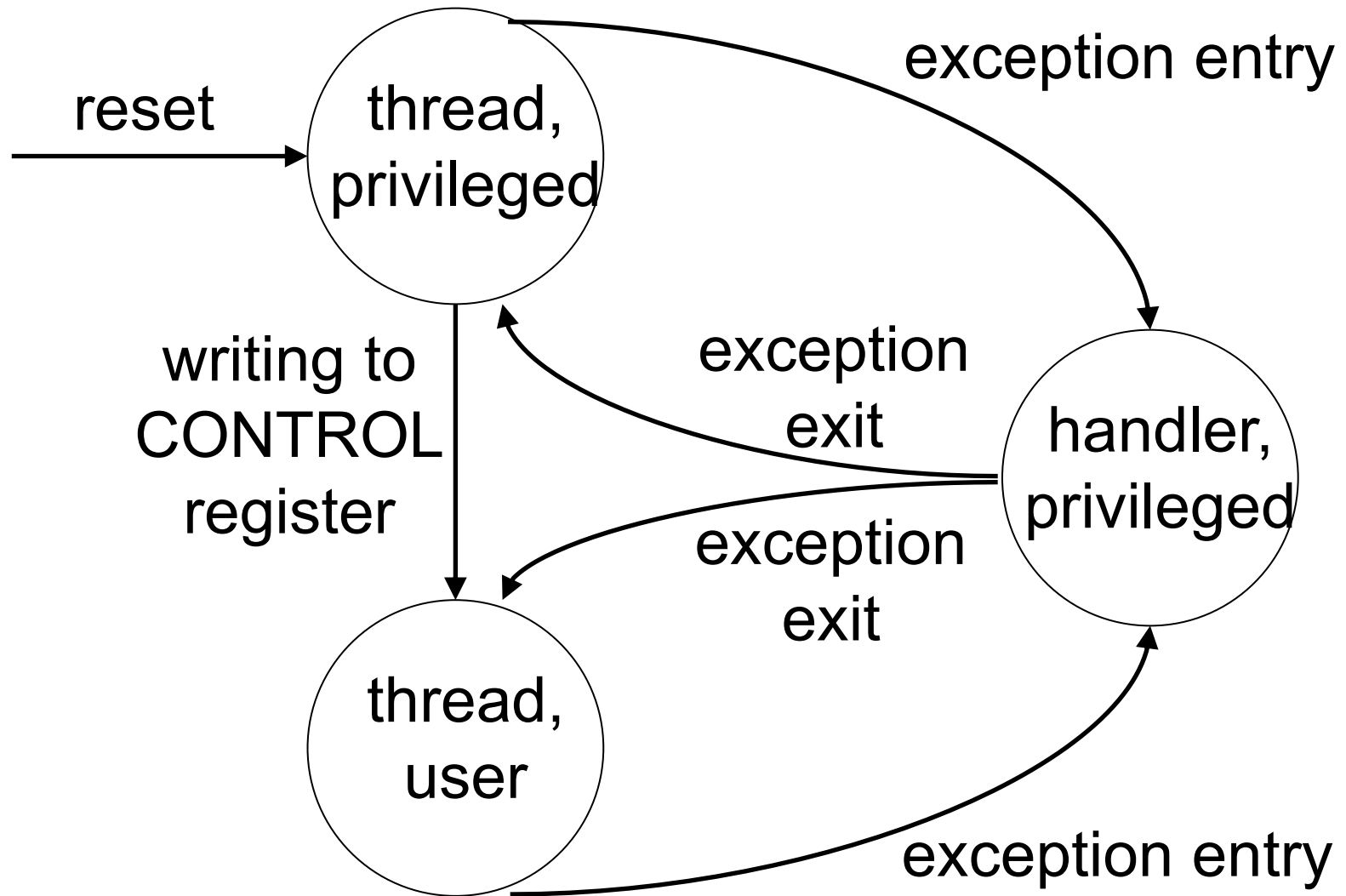
This work is licensed under the Creative Commons (CC BY-SA) License. To view a copy of this license, visit <http://creativecommons.org/licenses/by-sa/3.0/>



Processor modes

- Two operating modes:
 - thread mode: on reset or after completing the exception routine
 - handler mode: when an exception occurs.
- Two access levels:
 - user level: limited access to resources
 - privileged level: access to all resources.
- Handler mode is always privileged.

State diagram of processor modes



Control register

- CONTROL[0] changes the access level
 - 0: privileged level
 - 1: user level
- CONTROL[1] selects the current stack
 - 0: Main Stack Pointer (MSP)
 - 1: Process Stack Pointer (PSP)

	privileged level	user level
handler mode	CONTROL[0] = 0 CONTROL[1] = 0	not allowed
thread mode	CONTROL[0] = 0 CONTROL[1] = 0/1	CONTROL[0] = 1 CONTROL[1] = 0/1

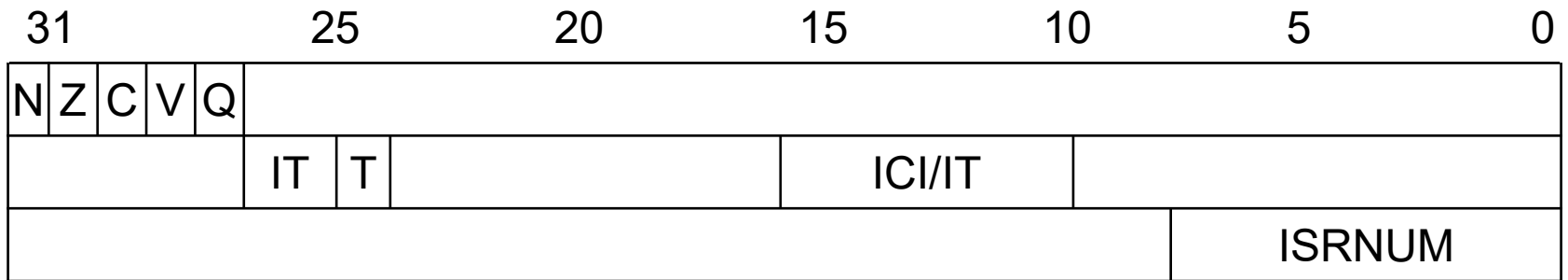
Switching access level

- CONTROL[0] is writable only in privileged level.
- Switching to user level can occur in any privileged code (Thread or Handler mode).
 - Example:

```
MRS  r8,  CONTROL
ORR  r8,  r8,  #1
MSR  CONTROL,  r8
```
- Switching to privileged level can occur only inside an exception handler (Handler mode).

Program Status Register

- It can be accessed as a combination of:
 - Application Program Status Register (APSR)
 - Execution Program Status Register (EPSR)
 - Interrupt Program Status Register (IPSR)
- When an exception occurs, IPSR stores the exception number.



Nested Vectored Interrupt Controller

- It is involved in the management of internal exceptions and up to 240 external interrupts.
- It contains control registers
 - interrupt processing
 - SYSTICK timer
 - debugging
- Registers are accessible as memory-mapped devices, starting from 0xE000E000.
- Most of registers are accessible only in privileged mode.

Exception sequence

Programmer's duty

- *relocate vector table**
- *set up priority level**
- enable the interrupt (only usage, bus and memory)
- implement exception handler

* *optional task*

Processor's duty

1. stack registers
2. fetch vector table
3. update registers
4. execute exception handler
5. unstack registers
6. update NVIC registers

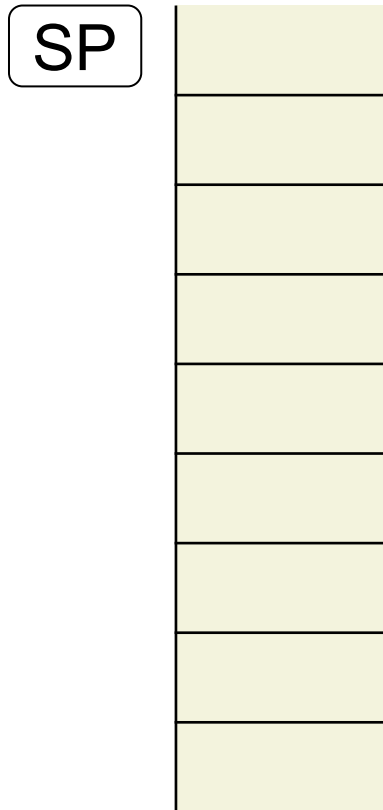
Task 1: stack registers

- `r0-r3`, `r12`, `LR`, `PC` and `PSR` are saved in the currently used stack (`MSP` or `PSP`)
- Then, `MSP` is always used during exception
- `PC` and `PSR` are stacked first, so instruction fetch and `PSR` update can be started early.

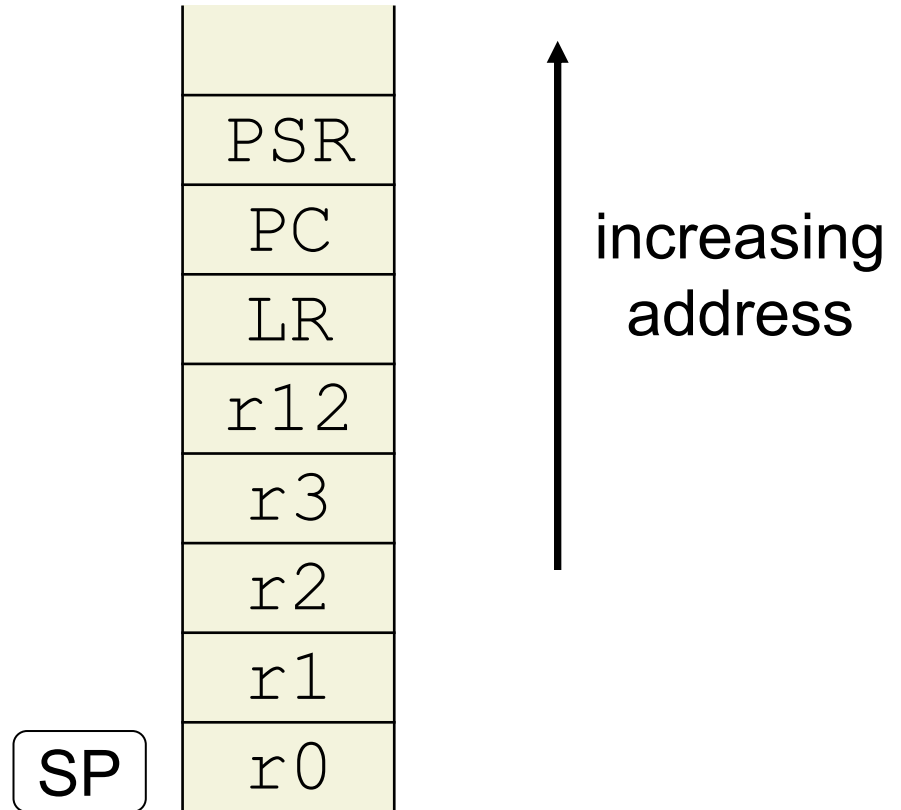
Register	<code>r0</code>	<code>r1</code>	<code>r2</code>	<code>r3</code>	<code>r12</code>	<code>LR</code>	<code>PC</code>	<code>PSR</code>
Stacking order	3	4	5	6	7	8	1	2
- Offset SP_{old}	32	28	24	20	16	12	8	4
Offset SP_{new}	0	4	8	12	16	20	24	28

Task 1: stack registers

before
the exception



when exception
is raised



Task 2: fetch interrupt vector table

- NVIC puts the exception code on the data bus
- The CPU uses the code as an index to access a vector named Interrupt Vector Table (IVT).
- Each entry of the IVT manages an exception: it contains either one ARM instruction or an address in memory.
- In the v7-M architecture, there are addresses of the exception service procedures.

Vector table

Exception type	Priority	Position	Vector address
(Top of Stack)		0	0x00000000
Reset	-3	1	0x00000004
NMI	-2	2	0x00000008
Hard fault	-1	3	0x0000000C
Memory management fault	programmable	4	0x00000010
Bus fault	programmable	5	0x00000014
Usage fault	programmable	6	0x00000018
Reserved		7-10	
Supervisor call (SVC)	programmable	11	0x0000002C
Debug monitor	programmable	12	0x00000030
Reserved		13	
Pendable Service Call	programmable	14	0x00000038
System Tick Timer	programmable	15	0x0000003C
External interrupts	programmable	16 - 255	≥0x00000040

Task 3: register updates

- The following registers are updated when entering the exception handler:
 - SP (either MSP or PSP): new value after stacking
 - PSR (IPSR only): exception number
 - PC: address of the exception handler
 - LR: exception return information EXC_RETURN
 - some NVIC registers: pending status of exception is cleared, active bit of exception is set.

EXC_RETURN

- Bit 0: processor state. 0 -> ARM, 1 -> Thumb
- Bit 1: reserved, it must be 0
- Bit 2: return stack. 0 -> MSP, 1 -> PSP
- Bit 3: return mode. 0 -> handler, 1 -> thread
- Bits 4-31: each bit must be 1
- Allowed values of EXC_RETURN are:
 - 0xFFFFFFF1: return to handler mode
 - 0xFFFFFFF9: return to thread mode with MSP
 - 0xFFFFFFF9D: return to thread mode with PSP

Task 5 and 6: exception exits

1. The interrupt return sequence is triggered
 - BX LR
 - PUSH {..., LR} and POP {..., PC}
 - STM xx, {..., LR} and LDM xx, {..., PC}
2. Unstacking: registers pushed to the stack are restored in the same order
3. NVIC register update
 - the active bit of the exception is cleared
 - if the input of an external interrupt is still asserted, the pending bit is set again, so handler is reentered

Task 4: execute exception handler

- Possible operations performed by the handler:
 - reading a fault status register (FSR) to know the cause of usage, bus, memory management fault
 - reading a fault address register (FAR) to know the memory location of a *precise* fault (bus or memory)
 - reading the stacked program counter to retrieve the offending instruction
- Value of bits can be checked through a mask.

```
LDR r0, =FSRaddress
```

```
LDR{size} r1, [r0]
```

```
TEQ r1, #mask
```

Fault status registers (FSR)

- Bits in a FSR indicate the cause of the fault
- Bits can be read or cleared by writing 1.

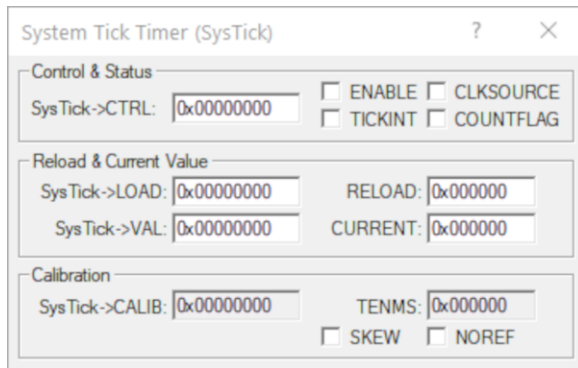
Register	Size	Address
Memory Management Fault Status register	byte	0xE000ED28
Bus Fault Status register	byte	0xE000ED29
Usage Fault Status register	half word	0xE000ED2A
Hard Fault Status register	word	0xE000ED2C
Debug Fault Status register	word	0xE000ED30
Auxiliary Fault Status register	word	0xE000ED3C

Other useful registers

Register	Type	Size	Address
Configuration Control Register	R/W	word	0xE000ED14
System Handler Control and State Register	R/W	word	0xE000ED24
Memory Management Fault Address Register (MMFAR)	R	word	0xE000ED34
Bus Fault Address Register (BFAR)	R	word	0xE000ED38

Useful debugging windows

- Open Peripherals -> Core Peripherals ->
 - Nested Vectored Interrupt Controller
 - System Tick Timer
 - Fault Reports



System Tick Timer (SysTick)

Control & Status

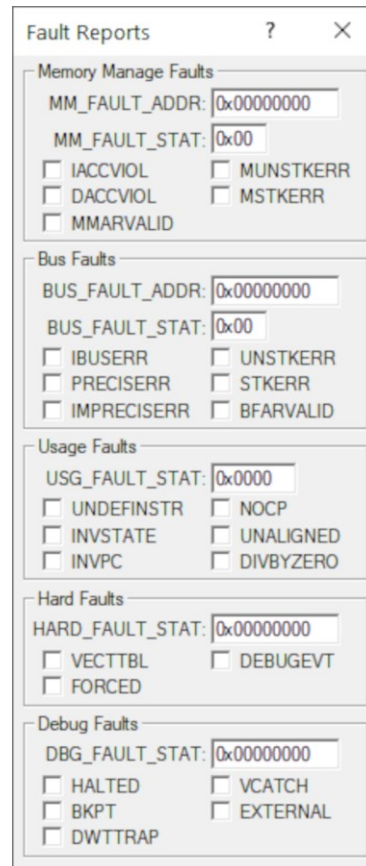
SysTick->CTRL: ☐ ENABLE ☐ CLKSOURCE ☐ TICKINT ☐ COUNTFLAG

Reload & Current Value

SysTick->LOAD: RELOAD:
SysTick->VAL: CURRENT:

Calibration

SysTick->CALIB: TENMS: ☐ SKEW ☐ NOREF



Fault Reports

Memory Manage Faults

MM_FAULT_ADDR: MM_FAULT_STAT:

☐ IACCVIOL ☐ MUNSTKERR
☐ DACCVIOL ☐ MSTKERR
☐ MMARVALID

Bus Faults

BUS_FAULT_ADDR: BUS_FAULT_STAT:

☐ IBUSERR ☐ UNSTKERR
☐ PRECISERR ☐ STKERR
☐ IMPRECISERR ☐ BFARVALID

Usage Faults

USG_FAULT_STAT:

☐ UNDEFINSTR ☐ NOCP
☐ INVSTATE ☐ UNALIGNED
☐ INVPC ☐ DIVBYZERO

Hard Faults

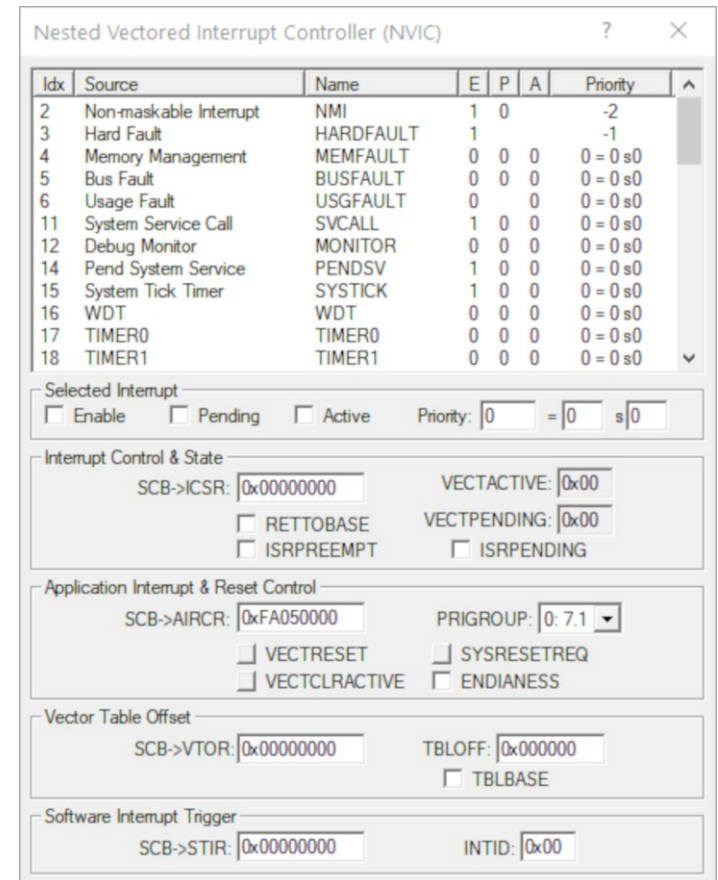
HARD_FAULT_STAT:

☐ VECTTBL ☐ DEBUGEVT
☐ FORCED

Debug Faults

DBG_FAULT_STAT:

☐ HALTED ☐ VCATCH
☐ BKPT ☐ EXTERNAL
☐ DWTTRAP



Nested Vectored Interrupt Controller (NVIC)

Idx	Source	Name	E	P	A	Priority
2	Non-maskable Interrupt	NMI	1	0		-2
3	Hard Fault	HARDFAULT	1			-1
4	Memory Management	MEMFAULT	0	0	0	0 = 0 s0
5	Bus Fault	BUSFAULT	0	0	0	0 = 0 s0
6	Usage Fault	USGFAULT	0		0	0 = 0 s0
11	System Service Call	SVCALL	1	0	0	0 = 0 s0
12	Debug Monitor	MONITOR	0	0	0	0 = 0 s0
14	Pend System Service	PENDSV	1	0	0	0 = 0 s0
15	System Tick Timer	SYSTICK	1	0	0	0 = 0 s0
16	WDT	WDT	0	0	0	0 = 0 s0
17	TIMER0	TIMER0	0	0	0	0 = 0 s0
18	TIMER1	TIMER1	0	0	0	0 = 0 s0

Selected Interrupt

☐ Enable ☐ Pending ☐ Active Priority: = s

Interrupt Control & State

SCB->ICSR: VECTACTIVE:
☐ RETTOBASE VECTPENDING:
☐ ISRPREEMPT ☐ ISRSPENDING

Application Interrupt & Reset Control

SCB->AICR: PRIGROUP:
☐ VECTRESET ☐ SYSRESETREQ
☐ VECTCLRACTIVE ☐ ENDIANESS

Vector Table Offset

SCB->VTOR: TBLOFF: ☐ TBLBASE

Software Interrupt Trigger

SCB->STIR: INTID:

System exception types

- Reset
- Non maskable interrupt (NMI)
- Hard fault
- Memory management fault
- Bus fault
- Usage fault
- Supervisor call (SVC)
- Debug monitor
- Pendable Service Call
- System Tick Timer (SYSTICK)

Non maskable interrupt (NMI)

- The use of NMI depends on the SoC design.
- For example, NMI can be connected to:
 - watchdog timer
 - voltage-monitoring block, which warns the CPU when the voltage drops below a certain level.
- The NMI exception can be activated any time, even right after the core exits reset.
- It has a fixed priority levels, equal to -2.
 - it has the highest priority among all exceptions, except Reset.

Hard Fault Status register

Bit	Possible causes of hard fault
31	Debug event (breakpoint or watchpoint).
30	A bus fault, memory management fault, or usage fault occurred, but the corresponding handler is disabled or cannot be started because another exception with same or higher priority is running, or because exception mask is set.
1	Failed interrupt vector fetch during exception processing (due to bus fault or incorrect vector table offset setup).

Hard fault example

- Attempting to switch to ARM state causes a usage fault.
- If usage fault is not enabled, a hard fault is generated.

```
ADRL r0, label
```

```
BX r0
```

System Handler Control and State

Bit	Type	Description
18	R/W	Usage fault handler enable
17	R/W	Bus fault handler enable
16	R/W	Memory management fault handler enable
15	R/W	SVC pended (replaced by higher-priority exception)
14	R/W	Bus fault pended
13	R/W	Memory management fault pended
12	R/W	Usage fault pended
11	R/W	Read as 1 if SYSTICK exception is active
10	R/W	Read as 1 if PendSV exception is active
8	R/W	Read as 1 if debug monitor exception is active
7	R/W	Read as 1 if SVC exception is active
3	R/W	Read as 1 if usage fault exception is active
1	R/W	Read as 1 if bus fault exception is active
0	R/W	Read as 1 if memory management fault is active

Enabling faults

```
LDR r2, =SystemHandlerControl
LDR r1, [r2]
; enable usage fault
ORR r1, #0x40000
; enable bus fault
ORR r1, #0x20000
; enable memory mngm. fault
ORR r1, #0x10000
; ORR r1, #0x70000 enable all
STR r1, [r2]
```

Usage Fault Status Register

Bit	Possible causes of usage fault
9	Division by zero and DIV_0_TRP is set.
8	Unaligned memory access attempted and UNALIGN_TRP is set
3	Attempt to execute a coprocessor instruction
2	Invalid EXC_RETURN during exception return. Invalid exception active status. Invalid value of stacked IPSR (stack corruption). Invalid ICI/IT bit for current instruction.
1	Branch target address to PC with LSB equals 0
0	Use of not supported (undefined) instruction.

Usage fault examples (1)

- switch to ARM state

```
ADRL r0, label
```

```
BX r0
```

- coprocessor instruction

```
LDC p1, c0, [r1]
```

- undefined instruction:

```
DCD 0xe7f0def0
```

within the executed code.

Note: exception raised only in hardware debug.

Usage fault examples (2)

- division by zero

```
LDR  r4, =ConfigurationControl
LDR  r3, [r4]
ORR  r3, #0x10
STR  r3, [r4]
MOV  r0, #0
MOV  r1, #0x11111111
UDIV r2, r1, r0
```

Usage fault examples (3)

- unaligned access

```
LDR r4, =ConfigurationControl
LDR r3, [r4]
ORR r3, #0x8
STR r3, [r4]
LDR r1, =myVar1
LDR r3, [r1] ;unaligned access?
LDR r2, =myVar2
LDR r4, [r2] ;unaligned access
stop      B stop
myVar1    DCB 0x11
myVar2    DCB 0x22
```

Configuration Control Register

Bit	Description
9	Force exception stacking start in double word aligned address.
8	Ignore data bus fault during hard fault and NMI.
4	Trap on divide by 0
3	Trap on unaligned accesses
1	If set to 1, allow user code to write to Software Trigger Interrupt register
0	If set to 1, allow exception handler to return to thread state at any level by controlling return value.

Bus Fault Status Register

Bit	Possible causes of bus fault	
7	Indicates that the Bus Fault Address Register (BFAR) contains a valid bus fault address (only precise violation)	
4	Stacking error: stack pointer is corrupted, or stack size became too large (reaching an undefined memory region), or PSP is used but not initialized	
3	Unstacking error or stack pointer changed during exception	
2	Imprecise data access violation (previously completed operation)	device not initialized, access of privileged-only resource in user mode, incorrect transfer size for the device.
1	Precise data access violation (last completed operation)	
0	Instruction access violation: branch to invalid region, invalid exception return code, invalid entry in interrupt vector table, access to NVIC in user mode.	

Bus fault examples (1)

- PSP is used but not initialized

```
MRS r8, CONTROL
```

```
ORR r8, r8, #2
```

```
MSR CONTROL, r8
```

```
PUSH {r1}
```

- To avoid the bus fault, before the PUSH add

```
LDR r0, =initial_psp
```

```
MSR PSP, r0
```

with this previous memory allocation:

```
AREA PSTACK, READWRITE, ALIGN=3
```

```
SPACE    Stack_Size
```

```
initial_psp
```

- Note: exception is raised only in hardware debug

Bus fault examples (2)

- access to a privileged memory location

```
MRS r8, CONTROL ;to user level
```

```
ORR r8, r8, #1
```

```
MSR CONTROL, r8
```

```
LDR r0, =0xE000ED2C
```

```
LDR r1, [r0]
```

- the data access violation is precise: the fault address can be read in BFAR

```
LDR r7, =BusFaultAddressRegister
```

```
LDR r2, [r7]
```

MemManage Status Register

Bit	Possible causes of memory management fault
7	Indicates that Memory Manage Address Register (MMAR) contains a valid fault addressing value
4	Stacking error: stack pointer is corrupted, or stack size became too large (reaching a region not defined by the MPU or disallowed in the MPU configuration)
3	Unstacking error or MPU configuration changed by exception handler.
1	Violation to memory access protection, which is defined by MPU setup. For example, user application trying to access privileged-only region.
0	Violation to memory access protection; branch to nonexecutable regions ; invalid exception return code; invalid entry in exception vector table.

Memory management fault example

- branch to nonexecutable region

```
LDR r0, =0x40000001
```

```
BX r0
```

- 0x40000000 is the starting address of the peripherals region.
- the least significant bit of the address must be 1 to avoid a usage fault.

Supervisor call

- A supervisor call exception is generated as:
`SVC #immediate`
- E.g.: the operating system provides access to hardware through a system function call `SVC`
- When the software exception handler in the operating system is executed, it provides the service requested by the user application.
- Benefits:
 - robustness: access to hardware is under OS control
 - portability: the user application does not need to know the programming details of the hardware.

Handler of supervisor call

- The handler can determine the immediate data value in the `SVC` instruction as follows:
 1. determining the stack used for stacking registers
 2. reading the stacked PC
 3. reading the instruction at the address pointed by the stacked PC
 4. masking out the unneeded bits (operating code)
- The `SVC` instruction is encoded in 16 bits, and the immediate value is stored in the least significant byte.

Implementation of SVC handler

```
;Test bit 2 of EXC_RETURN in LR
TST LR, #0x4
ITE EQ
MRSEQ r0, MSP
MRSNE r0, PSP
;get stacked PC from stack
LDR r1, [r0, #24] ;see slide 10
;get immediate from instruction
LDRB r0, [r1, #-2]
```

SYSTICK exception

- A 24-bit timer is used to generate interrupts
 - internal timer: core free-running clock
 - external timer: external reference clock (STCLK)
- Possible uses:
 - alarm timer
 - timing measurement
 - context switch among multiple tasks
- Controlled by four registers.

SYSTICK registers

SYSTICK Register	Size	Address
Control and Status Register	32 bits	0xE000E010
Reload Value Register	24 bits	0xE000E014
Current Value Register	24 bits	0xE000E018
Calibration Value Register	32 bits	0xE000E01C

- SYSTICK Reload Value Register stores the value to reload when timer reaches 0.
- SYSTICK Current Value Register stores the current value of the timer. Writing any number clears its content.

SYSTICK Control and Status Register

Bit	Type	Description
16	R	Read as 1 if counter reaches 0 since last time this register is read; clear to 0 when read or when current counter value (in Current Value Register) is cleared
2	R/W	1 = use processor free running clock 0 = use external reference clock (STCLK)
1	R/W	1 = generate interrupt when timer reaches 0 0 = Do not generate interrupt
0	R/W	1 = enable SYSTICK Timer 0 = disable SYSTICK Timer

SYSTICK Calibration Value Register

Bit	Type	Description
31	R	1 = external reference clock not available 0 = external reference clock available
30	R	1 = calibration value is not exactly 10 ms 0 = calibration value is accurate
23:0	R/W	Calibration value to use for 10 ms timings. Depending on the value of bit 30, this might be exactly 10ms or the closest value. If the calibration value is read as 0, then it is not known. This is probably because the reference clock is an unknown input from the system or scalable dynamically.

SYSTICK configuration

1. Stop timer counter to prevent interrupt triggered accidentally.
 - write 0 to the Control and Status register
2. Set the desired interval between interrupts
 - write a new value to the Reload Value register.
3. Reset the SYSTICK timer counter
 - write any value to the Current Value register
4. Start the SYSTICK timer
 - write proper value to Control and Status register.
 - e.g.: core clock, interrupt enabled, timer enabled

Code for SYSTICK configuration

```
LDR r0, =SYScontrolAndStatusReg
MOV r1, #0
STR r1, [r0] ; step 1
LDR r0, =SYSreloadValueReg
LDR r1, =1023 ; example
STR r1, [r0] ; step 2
LDR r0, =SYScurrentValueReg
STR r1, [r0] ; step 3
LDR r0, =SYScontrolAndStatusReg
MOV r1, #7
STR r1, [r0] ; step 4
```